

Going through report: (Remove huge title)
(Remove extract)
(Minimize margin)

- Expand on
- Remove
- Random shit
- I don't understand
- Very cool

1.1 a) No need for equivalent one line argument

Main

So S never changes, and $S(t) = S(0)$ for all t . ~~The same fact reads off the matrix directly: every column of M sums to one, which is to say $\mathbf{1}^T M = \mathbf{1}^T$ with $\mathbf{1} = (1, 1)^T$, and therefore $\mathbf{1}^T P(t+1) = \mathbf{1}^T P(t)$.~~
~~A column-stochastic update preserves total mass; here that mass is simply the number of organisms, conserved because conversion only moves individuals between types.~~

1.6)

Main

1.2 (b) Eigenvalues and eigenvectors

Because M is lower triangular its eigenvalues are its diagonal entries, $\lambda_1 = 1$ and $\lambda_2 = 1 - \epsilon$, which we confirm from $\det(M - \lambda I) = (1 - \epsilon - \lambda)(1 - \lambda) = 0$. Solving $(M - \lambda_1 I)v_1 = 0$ gives the directions. For $\lambda_1 = 1$ the first equation forces the first component to zero, leaving $v_1 = (0, 1)^T$. For $\lambda_2 = 1 - \epsilon$ we get $x + y = 0$, so $v_2 = (1, -1)^T$. ~~The first eigenvector is the all-type 2 state that the dynamics converges to, the second is the transient direction along which type 1 decays.~~

• include matrices visually. Show solving

1.2)

1.3 (c) Writing the initial state in the eigenbasis

With $P(0) = c_1 v_1 + c_2 v_2 = (c_2, c_1 - c_2)^T$, matching components gives $c_2 = p_1(0)$ and $c_1 = p_2(0) + p_1(0) = S(0)$. So c_1 is exactly the conserved total from part (a), which is a useful consistency check rather than a coincidence. Taking the natural normalisation $p_1(0) + p_2(0) = 1$ with $p_1(0) = \frac{1}{2}$ gives the tidy decomposition $P(0) = v_1 + \frac{1}{2} v_2$. ~~We find that the original notes get $c_1 = 1$ without stating this normalisation; the general coefficients above make the assumption explicit.~~

1.4)

1.4 (d) Closed-form evolution

The eigenbasis turns repeated matrix multiplication into ordinary powers. Since $M^t v_i = \lambda_i^t v_i$,

$$P(t) = M^t P(0) = c_1 \lambda_1^t v_1 + c_2 \lambda_2^t v_2 = c_1 v_1 + c_2 (1 - \epsilon)^t v_2. \quad \text{Show more steps...}$$

~~Each mode evolves on its own, scaled by its eigenvalue raised to the t . This is the whole advantage of diagonalising: a coupled recursion becomes two independent scalar sequences.~~

1.5)

1.5 (e) Long-run behaviour

Because $0 < 1 - \epsilon < 1$, the factor $(1 - \epsilon)^t \rightarrow 0$, so the transient mode dies and

$$P(t) \rightarrow c_1 v_1 = (0, p_1(0) + p_2(0))^T. \quad \text{Show steps...}$$

All mass ends up as type 2. Structurally this is inevitable: the second column of M is $(0, 1)^T$, so type 2 is absorbing, while type 1 leaks away at rate ϵ every step and is never replenished. The eigenvalue $\lambda_1 = 1$ carries the surviving steady state and $\lambda_2 < 1$ governs how quickly the rest drains into it.

There are two subtasks for task e

1.6)

1.6 (f) Checking (d) and (e) numerically

We verified the closed form against a direct simulation that uses no analytical input: iterate $P(t+1) = M P(t)$ for $T = 100$ generations at $\epsilon = 0.1$, and compare with the formula from (d). The two agree to the last bit of double precision, with maximum discrepancy of order 10^{-16} , and the trajectory settles on $(0, 1)$ as predicted. The assertion is what makes this a test rather than a demonstration: if a later edit broke the formula, the cell would stop rather than quietly plot the wrong thing.

```
P = np.zeros((T+1, 2)); P[0] = p0 # track A: brute-force iteration
for s in range(T): P[s+1] = M @ P[s]
P_closed = c1*v1 + c2*((1-eps)**t)[:None]*v2 # track B: closed form from (d)
assert np.max(np.abs(P - P_closed)) < 1e-10 # must agree to machine precision
```

Figure 1 shows the result. On the linear axis both populations move monotonically to their limits; on the semi-logarithmic axis the numeric $p_1(t)$ falls exactly along the line $\frac{1}{2}(1 - \epsilon)^t$, and a straight line there is the signature of geometric decay at rate $1 - \epsilon$.

1.7)

1.7 (g) A conversion rate that fades over time

Now let the rate decay, $\epsilon(t) = \epsilon_0 e^{-t/t_0}$, so the update matrix $M(t)$ is different at every step. The key observation is that the eigenvectors do not depend on ϵ , so v_1 and v_2 are shared by every $M(t)$: $M(t)v_1 = v_1$ and $M(t)v_2 = (1 - \epsilon(t))v_2$. The state is the ordered product $P(t) = M(t-1) \cdots M(0)P(0)$, and acting on $P(0) = c_1 v_1 + c_2 v_2$ the persistent mode is multiplied by 1 each step while the transient mode collects one factor $(1 - \epsilon(t))$ per step. Hence

$$P(t) = c_1 v_1 + c_2 \left[\prod_{s=0}^{t-1} (1 - \epsilon_0 e^{-s/t_0}) \right] v_2. \quad \text{Show steps}$$

The geometric power $(1 - \epsilon)^t$ of part (d) is replaced by a product, not by $(1 - \epsilon(t))^t$, which is the form the notes had recorded and which would only be right for a fixed matrix. The difference matters. Because

$\sum_{s=0}^{\infty} \epsilon_0 e^{-s/t_0}$ converges, the infinite product tends to a strictly positive limit Π_{∞} , so the transient mode does not decay to zero. The conversion effectively switches off as $\epsilon(t) \rightarrow 0$, freezing a residue $c_2 \Pi_{\infty}$ in the v_2 direction. The population therefore settles at $P(\infty) = c_1 v_1 + c_2 \Pi_{\infty} v_2$, and type 1 is never fully depleted. This was the result we were most keen to check, since it overturns the intuition built in parts (a)-(c).

1.8)

1.8 (h) Checking the fading rate

We validated the product formula exactly as in (f), iterating with a freshly built $M(s)$ at each step ($\epsilon_0 = 0.5$, $t_0 = 10$) and comparing against the closed form; agreement is again at the 10^{-16} level. Figure 2 overlays the fading-rate trajectory on the constant-rate case. The contrast is the whole point: with constant ϵ , p_1 heads for zero in a straight log-line, whereas with $\epsilon(t)$ it bends and locks onto a positive plateau once the rate has effectively vanished. The measured residue matches the small-rate estimate $\frac{1}{2} e^{-\epsilon_0/t_0}$ to the expected accuracy.

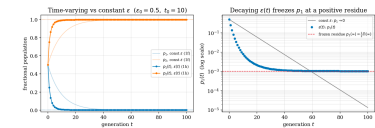


Figure 2: Fading rate $\epsilon(t) = \epsilon_0 e^{-t/t_0}$ (solid) against constant ϵ (faded). Right panel, log axis: the constant case drains to zero, while the fading case freezes at the positive residue $c_2 \Pi_{\infty}$ (dashed).